Engine de Simulação do Problema dos Três Corpos

José Henrique Noal[1]

[1]Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul - Câmpus Farroupilha
Farroupilha, Rio Grande do Sul

joseh.noal@gmail.com,

rogerio.xavier@farroupilha.ifrs.edu.br

Abstract. *The following paper describes a project for a physics engine. Focusing on a well-known physics PROBLEM: The N-body problem, which describes any physical system where the bodies present are at a distance, WITH mass and velocity sufficient to interact with each other. That means that the sole existence of the bodies is enough to create interaction. This interaction can vary according to the number of bodies: having two bodies will make the system tend to order, but having more bodies will make the system tend toward chaos. The focus of the engine is to be able to deduce and calculate the integrals of the system efficiently, using a wide range of algorithms, such as Bayres-Hut and parallel execution.*
Keywords: *N-body problem, physics engine, Barnes-Hut algorithm, numerical simulation.*

Resumo. *Este artigo descreve o projeto de uma engine de simulação. Com foco em um problema conhecido da física: O problema de N-corpos, que descreve qualquer sistema em que os corpos presentes têm a distância, massa e velocidade suficientes para causar interação. Essa interação pode variar dependendo da quantidade de corpos: tendo dois corpos ou menos fará com que o sistema tenda ao equilíbrio, porém, tendo mais corpos, fará o sistema tender ao caos. O foco dessa engine é possibilitar a dedução e o cálculo as integrais do sistema de maneira eficaz, utilizando diversos algoritmos, como o Barnes-Hut e execuções paralelas.*
Palavras chave: *Problema de N-Corpos, engine física, Barnes-Hut algoritmo, simulação numérica.*

1. Introdução

O problema de N-corpos é um problema clássico da física, presente em áreas como a mecânica celestial, a astrofísica computacional e a dinâmica de fluidos (Carnegie Mellon University, 2026; OLEGáRIO, 2024; Instituto Nacional de Pesquisas Espaciais, 2026). Ele se manifesta em situações em que múltiplos corpos possuem massas comparáveis e se encontram a distâncias suficientemente próximas para que ocorram interações gravitacionais relevantes. Nesses casos, o sistema resultante é descrito por um conjunto de equações diferenciais acopladas, para as quais não há solução analítica geral em sistemas com três ou mais corpos. Assim, a análise dessa situação-problema depende fortemente do uso de simulações computacionais e métodos numéricos (Carnegie Mellon University, 2026; SPRINGEL, 2005).

Dado o contexto do tema, define-se como objetivo desenvolver uma *engine* de simulação física para o problema de N-corpos, utilizando técnicas de otimização algorítmica e paralelização, a fim de avaliar o impacto dessas abordagens no desempenho computacional e na previsibilidade de sistemas dinâmicos potencialmente caóticos.

Objetivos específicos:

- Implementar um modelo numérico para simular o problema de N-corpos baseado na interação gravitacional entre partículas.
- Implementar e integrar uma estrutura hierárquica de decomposição espacial baseada em métodos aproximativos para reduzir a complexidade computacional do cálculo das interações entre corpos (BARNES; HUT, 1986; Princeton University, 2026).
- Investigar e implementar estratégias de paralelização e distribuição do processamento da simulação (University of North Carolina, 2026; KALE; KRY, 2024).
- Projetar uma plataforma de execução que permita a visualização dos resultados tanto em ambiente local quanto, posteriormente, em ambiente web.
- Implementar mecanismos de parametrização da simulação por meio de entrada de dados estruturada, como arquivos JSON, CSV e interface gráfica (JSON, 2026).
- Realizar experimentos comparativos para avaliar desempenho, escalabilidade e divergência de trajetórias em sistemas sensíveis às condições iniciais.

Diante desse cenário, torna-se necessária a investigação de técnicas que permitam reduzir o custo computacional dessas simulações sem comprometer significativamente sua precisão. Métodos aproximativos, como abordagens baseadas em decomposição espacial hierárquica, possibilitam reduzir o número de interações calculadas diretamente, substituindo parte dos cálculos individuais por aproximações entre grupos de corpos (ArborJS, 2026; Princeton University, 2026). Além disso, o avanço da disponibilidade de recursos computacionais distribuídos e paralelos abre novas possibilidades para a execução de simulações físicas em larga escala (University of North Carolina, 2026; SPRINGEL, 2005; KALE; KRY, 2024).

No entanto, a integração entre modelos físicos, técnicas de aproximação e estratégias de paralelização ainda representa um desafio relevante, especialmente em contextos educacionais e experimentais, nos quais se busca compreender o impacto dessas abordagens no desempenho, na escalabilidade e na estabilidade numérica das simulações. Justifica-se, assim, o desenvolvimento de uma plataforma de simulação que permita visualizar e avaliar diferentes estratégias de otimização e paralelização aplicadas ao problema

de N-corpos. Tal abordagem possibilita a análise do custo computacional envolvido e a investigação da sensibilidade às condições iniciais e da previsibilidade de sistemas dinâmicos potencialmente caóticos, contribuindo para a compreensão prática de conceitos fundamentais de física computacional e sistemas distribuídos.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico, abordando os principais conceitos relacionados ao problema de N-corpos, métodos numéricos, otimização algorítmica, decomposição espacial, paralelização e trabalhos semelhantes que envolvem simulações físicas computacionais. Em seguida, a Seção 3 descreve os procedimentos metodológicos adotados no desenvolvimento da proposta, incluindo as etapas de implementação, experimentação e análise dos resultados. A Seção **??** apresenta as tecnologias selecionadas para o desenvolvimento da *engine*, incluindo o núcleo de simulação *headless*, a exportação de dados e as possibilidades de visualização local ou na web. A Seção 4 descreve a modelagem do sistema, contemplando os principais artefatos utilizados para representar sua estrutura e comportamento. Posteriormente, a Seção **??** detalha a implementação da solução proposta e a Seção **??** apresenta os experimentos realizados, bem como a análise dos resultados obtidos. Por fim, a Seção **??** apresenta as considerações finais, limitações do trabalho e possibilidades de trabalhos futuros. As referências bibliográficas utilizadas são apresentadas ao final do artigo, seguidas dos apêndices necessários.

2. Referencial teórico

2.1. Problema de N-corpos

O problema de N-corpos constitui um dos temas clássicos da física computacional, sendo amplamente estudado em áreas como mecânica celestial, astrofísica e dinâmica molecular (Carnegie Mellon University, 2026; OLEGáRIO, 2024; Instituto Nacional de Pesquisas Espaciais, 2026). De modo geral, o problema consiste em determinar numericamente a evolução temporal de um sistema composto por múltiplas partículas que interagem entre si, geralmente por meio de forças dependentes da distância, como a força gravitacional.

A necessidade de simular a evolução desses sistemas levou ao desenvolvimento de diversos métodos computacionais voltados à redução do custo de simulação e ao aumento da escalabilidade (SPRINGEL, 2005). Em sistemas com muitos corpos, pequenas variações nas condições iniciais também podem gerar trajetórias significativamente diferentes ao longo do tempo, tornando o problema relevante tanto do ponto de vista computacional quanto experimental.

2.2. Abordagem direta e complexidade computacional

Uma abordagem direta para a resolução do problema consiste no cálculo explícito das interações gravitacionais entre todos os pares de partículas. Considerando um sistema com N corpos, a posição do corpo i pode ser representada por $\mathbf{r}_i(t)$, sua massa por m_i e a constante gravitacional por G. A evolução do movimento desse corpo pode ser descrita pela seguinte equação diferencial:

$$\frac{d^2\mathbf{r}_i}{dt^2} = G \sum_{\substack{j=1 \\ j \neq i}}^{N} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\|\mathbf{r}_j - \mathbf{r}_i\|^3} \tag{1}$$

Nessa formulação, a aceleração de cada corpo depende da soma das contribuições gravitacionais exercidas por todos os demais corpos do sistema. Portanto, para cada passo da simulação, é necessário calcular as interações entre os corpos i e j, com $i \neq j$. Esse processo resulta em complexidade computacional quadrática, representada por $O(N^2)$, em que N corresponde ao número de corpos simulados (Carnegie Mellon University, 2026; GREENGARD, 1990; BARNES; HUT, 1986; BRANDT, 2022).

Essa característica limita a viabilidade de simulações com grandes quantidades de elementos, pois o número de cálculos cresce rapidamente conforme novos corpos são adicionados. Como alternativa, métodos aproximativos buscam reduzir o número de interações calculadas diretamente, permitindo que simulações maiores sejam executadas com menor custo computacional.

A notação $O(N \log N)$ é utilizada na análise de complexidade computacional para representar algoritmos cujo custo cresce de forma quase linear em relação ao tamanho da entrada. No contexto do problema de N-corpos, métodos hierárquicos como o algoritmo de Barnes-Hut reduzem o custo da simulação ao agrupar corpos distantes e aproximar sua influência gravitacional por meio do centro de massa, permitindo que o cálculo das forças seja realizado em aproximadamente $O(N \log N)$ (BARNES; HUT, 1986; ArborJS, 2026; Princeton University, 2026; BRANDT, 2022).

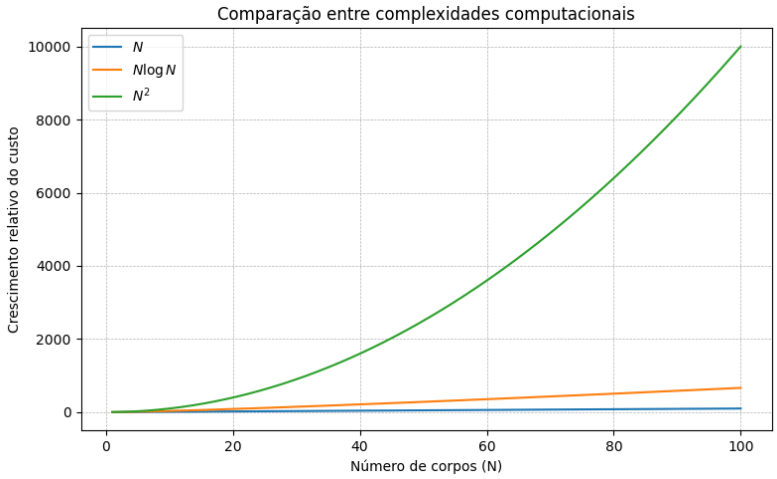


Figura 1. Comparação entre o crescimento das complexidades $O(N^2)$ e $O(N \log N)$. Fonte: Autoria própria, 2026

2.3. Algoritmo de Barnes-Hut

Entre os métodos aproximativos conhecidos destaca-se o algoritmo de Barnes-Hut, que utiliza uma estrutura hierárquica de decomposição espacial para organizar os corpos simulados. A ideia central do método é substituir interações individuais entre corpos distantes por aproximações baseadas no centro de massa de um grupo de partículas, reduzindo o número total de cálculos necessários (BARNES; HUT, 1986; ArborJS, 2026; Princeton University, 2026).

Nesse algoritmo, regiões do espaço são subdivididas recursivamente de acordo com a distribuição dos corpos. Durante o cálculo das forças, o sistema decide se um agrupamento de corpos pode ser tratado como uma única massa aproximada ou se é necessário

analisar suas subdivisões internas. Essa decisão depende de um critério de aproximação que considera a distância entre o corpo analisado e a região agrupada (BARNES; HUT, 1986; Princeton University, 2026).

2.4. Quadtrees e octrees

Quadtrees e octrees são estruturas hierárquicas de decomposição espacial utilizadas para organizar objetos com base em sua posição no espaço. Uma *quadtree* divide recursivamente uma região bidimensional em quatro sub-regiões, enquanto uma *octree* realiza processo análogo em três dimensões, subdividindo o espaço em oito regiões (ArborJS, 2026; Princeton University, 2026). Como visto na figura:

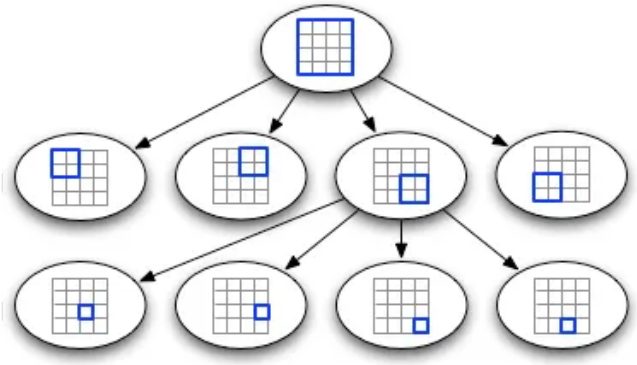


Figura 2. Exemplo de decomposição espacial utilizando uma quadtree. Fonte: (Adriano Croco, 2026)

No contexto do algoritmo de Barnes-Hut, essas estruturas permitem agrupar partículas espacialmente distantes e representar conjuntos de corpos por meio de uma única massa aproximada. Em simulações bidimensionais, a quadtree é suficiente para organizar o espaço simulado; já em simulações tridimensionais, a octree é mais adequada por representar a subdivisão espacial em três eixos (BARNES; HUT, 1986; ArborJS, 2026; Princeton University, 2026).

2.5. Paralelização e distribuição

Além das técnicas de otimização algorítmica, pesquisas em computação de alto desempenho têm explorado paralelização e distribuição do processamento em simulações de N-corpos. Modelos baseados em decomposição de domínio espacial permitem dividir a carga computacional entre múltiplos processos ou nós, reduzindo o tempo de execução e possibilitando melhor aproveitamento de arquiteturas paralelas e distribuídas (SPRINGEL, 2005; BRANDT, 2022; University of North Carolina, 2026).

Entretanto, tais abordagens introduzem desafios adicionais relacionados à sincronização de dados, comunicação entre processos e à estabilidade numérica das simulações. Em sistemas distribuídos, é necessário coordenar a troca de informações entre regiões distintas da simulação, especialmente quando corpos localizados em diferentes domínios espaciais ainda exercem influência uns sobre os outros.

2.6. Ferramentas e trabalhos relacionados

Ferramentas científicas e bibliotecas especializadas, como ambientes voltados à astrofísica computacional e *frameworks* de simulação dinâmica, demonstram a relevância prática do problema e evidenciam a necessidade de soluções que conciliem eficiência computacional, precisão numérica e flexibilidade de uso (SPRINGEL, 2005; TEUBEN, 2026; REIN; LIU, 2011). Esses trabalhos servem como referência para compreender diferentes estratégias de implementação, organização de simulações e análise de desempenho.

Nesse contexto, o desenvolvimento de plataformas experimentais de simulação permite investigar o impacto de diferentes técnicas de aproximação e paralelização sobre o desempenho e a previsibilidade de sistemas dinâmicos complexos. A proposta deste trabalho se insere nesse cenário ao buscar implementar de forma modular e extensível de uma *engine* física voltada à simulação do problema de N-corpos.

2.7. Hipótese da proposta

Com base nos conceitos apresentados, este trabalho parte da hipótese de que é possível reduzir significativamente o custo computacional da simulação do problema de N-corpos por meio da aplicação de técnicas de decomposição espacial hierárquica e paralelização.

Espera-se que a plataforma permita avaliar, de forma experimental, o desempenho computacional, a escalabilidade e a divergência de trajetórias em sistemas sensíveis às condições iniciais. Além disso, a separação entre núcleo de simulação, visualização e exportação de dados busca facilitar a evolução futura do sistema para diferentes ambientes de execução.

3. Procedimentos Metodológicos

Este trabalho caracteriza-se como uma pesquisa aplicada, de caráter experimental, pois tem como finalidade desenvolver uma solução computacional para simulação do problema de N-corpos e avaliar seu comportamento sob diferentes estratégias de execução. A proposta será conduzida por meio do desenvolvimento incremental de uma *engine* física modular, permitindo comparar uma abordagem direta por força bruta com outra otimizada baseada em decomposição espacial hierárquica.

Inicialmente, será realizada a consolidação do referencial teórico sobre o problema de N-corpos, métodos numéricos, complexidade computacional, algoritmo de Barnes-Hut, estruturas espaciais e estratégias de paralelização. Essa etapa tem como objetivo fundamentar as escolhas técnicas adotadas durante o desenvolvimento da solução.

Em seguida, será desenvolvida a primeira versão do núcleo de simulação, implementando a abordagem direta para o cálculo das interações gravitacionais entre todos os pares de corpos. Essa versão servirá como base de validação do modelo físico e como referência para comparação com as otimizações posteriores. Após essa etapa, será implementado o mecanismo de exportação dos estados da simulação em formato estruturado, permitindo registrar a evolução temporal dos corpos simulados e reutilizar esses dados em ferramentas de análise ou visualização.

Posteriormente, será implementada uma estrutura de decomposição espacial baseada em *quadtree*, permitindo a integração do algoritmo de Barnes-Hut ao motor de simulação. Essa etapa tem como objetivo reduzir o custo computacional do cálculo das

forças, substituindo parte das interações individuais por aproximações baseadas no centro de massa de agrupamentos de corpos (BARNES; HUT, 1986; ArborJS, 2026; Princeton University, 2026).

Após a implementação das duas abordagens, serão realizados experimentos comparativos para avaliar o desempenho computacional das simulações. Os testes deverão considerar diferentes quantidades de corpos simulados, tempo de execução por etapa, custo computacional e comportamento das trajetórias geradas. Caso o cronograma permita, também será investigada uma estratégia inicial de paralelização ou de distribuição do processamento, com o objetivo de analisar seus impactos na escalabilidade da solução.

Por fim, os resultados obtidos serão analisados de forma comparativa, considerando o desempenho da abordagem direta e da abordagem otimizada. A análise buscará verificar se a utilização de decomposição espacial hierárquica e paralelização contribui para a redução do custo computacional da simulação, bem como observar a divergência de trajetórias em sistemas sensíveis às condições iniciais.

4. Desenvolvimento

O núcleo da simulação será desenvolvido na linguagem de programação Odin, escolhida por sua simplicidade sintática, proximidade com a programação de baixo nível e foco em desempenho (Odin, 2026b, 2026a). A proposta prevê o desenvolvimento de uma *engine* física estruturada como um núcleo *headless*, isto é, desacoplado de uma interface gráfica específica. Nesse modelo, a lógica principal da simulação pode ser executada independentemente da camada de visualização, permitindo que seus resultados sejam consumidos por diferentes interfaces ou ferramentas externas (Talon.One, 2026).

Essa abordagem foi escolhida para favorecer a modularidade e a reutilização do sistema. Inicialmente, a simulação deverá ser executada em ambiente local, com uma visualização simples voltada à validação dos algoritmos e à observação do comportamento dos corpos simulados. Posteriormente, como o núcleo da simulação não estará diretamente acoplado à interface, os dados gerados poderão ser consumidos por outros visualizadores, incluindo uma possível aplicação web.

Para registrar os estados da simulação ao longo do tempo, será utilizado o formato JSON Lines (JSONL), que permite armazenar cada passo temporal como um registro independente e estruturado (JSON, 2026; JSON Lines, 2026). Essa escolha facilita a exportação, a análise e a reprodução dos dados simulados, além de reforçar a separação entre o motor físico e a camada de apresentação.

4.1. Modelagem

4.1.1. Análise e Classificação dos Requisitos

Os requisitos funcionais representam as funcionalidades que o sistema deverá disponibilizar para permitir a configuração, a execução, a visualização e a análise das simulações.

- RF01: Definir sistema de corpos: o sistema deve permitir a criação de um conjunto de corpos com propriedades físicas, como posição, velocidade e massa.
- RF02: Inserir parâmetros da simulação: o sistema deve permitir a inserção de parâmetros de simulação por meio de diferentes formatos, como interface gráfica, arquivos JSON e *Comma-Separated Values* (CSV).
- RF03: Executar simulação: o sistema deve executar a simulação do problema de N-corpos ao longo do tempo, atualizando o estado dos corpos a cada passo temporal.
- RF04: Calcular interações físicas: o sistema deve calcular as forças de interação entre os corpos com base no modelo físico definido.
- RF05: Aplicar métodos de otimização: o sistema deve permitir a execução da simulação utilizando métodos de otimização para a reduzir a complexidade computacional.
- RF06: Visualizar simulação: o sistema deve permitir visualizar a evolução do sistema de partículas ao longo do tempo.
- RF07: Exportar dados: o sistema deve permitir a exportação dos dados da simulação em formato estruturado para análise posterior.
- RF08: Comparar abordagens: o sistema deve permitir a comparação entre diferentes métodos de cálculo de forças quanto ao desempenho e à precisão dos resultados.

Os requisitos não funcionais descrevem características de qualidade esperadas para o sistema, relacionadas ao desempenho, a portabilidade, à modularidade e a extensibilidade.

- RNF01: Desempenho computacional: o sistema deve buscar desempenho suficiente para executar simulações com múltiplos corpos em tempo adequado para a análise experimental.
- RNF02: Suporte à execução paralela: a arquitetura do sistema deve permitir a execução em paralelo de múltiplos processos ou *threads* para melhorar o desempenho da simulação.
- RNF03: Portabilidade: o sistema deve ser projetado de modo a possibilitar a execução em ambiente local e, posteriormente, a integração com o ambiente *web*.
- RNF04: Modularidade: o núcleo da simulação deve ser desacoplado da camada de visualização, permitindo a substituição ou extensão de componentes sem impactar toda a aplicação.
- RNF05: Extensibilidade: o sistema deve permitir a inclusão futura de novos métodos de cálculo, estratégias de otimização e formas de visualização.
- RNF06: Reprodutibilidade: os dados gerados pela simulação devem poder ser exportados em formato estruturado, permitindo análise posterior e repetição dos experimentos.

4.1.2. Casos de Uso

O diagrama de casos de uso apresentado na Figura 3 representa as principais interações entre o usuário e a *engine* de simulação proposta. O ator principal é o usuário, responsável por configurar, iniciar, acompanhar e exportar os dados produzidos pela simulação do problema de N-corpos.

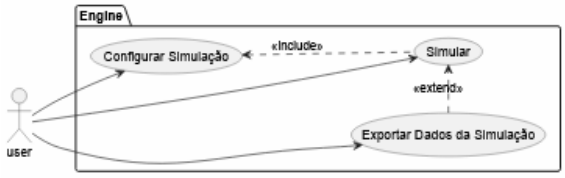


Figura 3. Diagrama de casos de uso. Fonte: Autoria própria, 2026, 2026

O caso de uso *Configurar simulação* representa a etapa em que o usuário define os parâmetros necessários para a execução da simulação, como a quantidade de corpos, massas, posições iniciais, velocidades e o método de cálculo a ser utilizado. Essa etapa é necessária para que o sistema receba as condições iniciais adequadas antes do início do processamento.

O caso de uso *Executar simulação* corresponde à realização da simulação propriamente dita. Nessa etapa, o sistema calcula as interações gravitacionais entre os corpos, atualiza suas velocidades e posições ao longo do tempo e mantém o estado da simulação a cada passo temporal. Esse caso de uso representa o núcleo funcional da aplicação.

O caso de uso *Visualizar resultados* permite acompanhar a evolução do sistema de partículas durante ou após a execução da simulação. Essa visualização tem como objetivo auxiliar na validação do comportamento físico simulado e facilitar a observação de padrões, aproximações e divergências entre trajetórias.

Por fim, o caso de uso *Exportar dados* permite que os estados gerados pela simulação sejam armazenados em formato estruturado, possibilitando análise posterior, reprodução dos experimentos e comparação entre diferentes métodos de cálculo. Essa funcionalidade contribui para a avaliação experimental do desempenho e da precisão das abordagens implementadas.

4.1.3. Comportamento

O diagrama de comportamento apresentado na Figura 4 representa os principais estados previstos para a execução da *engine* de simulação. Esse diagrama tem como objetivo descrever o ciclo de vida da simulação, desde a inicialização até a exportação dos dados gerados.

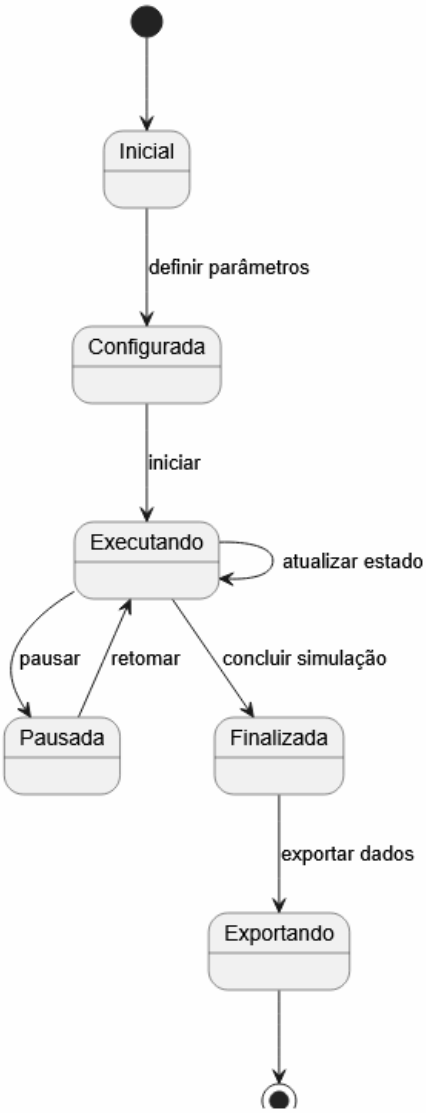


Figura 4. Diagrama de comportamento. Fonte: autoria própria, 2026.

O estado *Inicial* representa a abertura da *engine* e o momento anterior à configuração da simulação, no qual ainda serão definidos os dados e os parâmetros iniciais necessários para a execução.

O estado *Configurada* representa o momento em que os parâmetros iniciais já foram definidos, tornando a *engine* apta a iniciar o processo de simulação.

O estado *Executando* ocorre durante a execução da simulação, quando o sistema calcula das interações físicas entre os corpos, atualiza seus estados e apresenta os resultados ao usuário.

O estado *Pausada* representa uma interrupção temporária da execução, permitindo que o usuário suspenda a simulação caso seja necessário observar o estado atual ou realizar alguma ação antes de retomá-la.

O estado *Finalizada* indica o término da simulação, quando o processo de

atualização dos corpos é encerrado.

Por fim, o estado *Exportando* representa a etapa em que os dados produzidos pela simulação são organizados e disponibilizados ao usuário em formato estruturado para análise posterior.

4.1.4. Sequência

O diagrama de sequência apresentado na Figura 5 descreve a interação entre os principais componentes da *engine* durante a configuração, execução, visualização e exportação de uma simulação. O fluxo inicia com o usuário informando os parâmetros da simulação por meio da interface, que encaminha esses dados ao módulo de configuração para validação. Após a confirmação dos parâmetros, o usuário inicia a simulação, acionando o motor responsável por coordenar o processo.

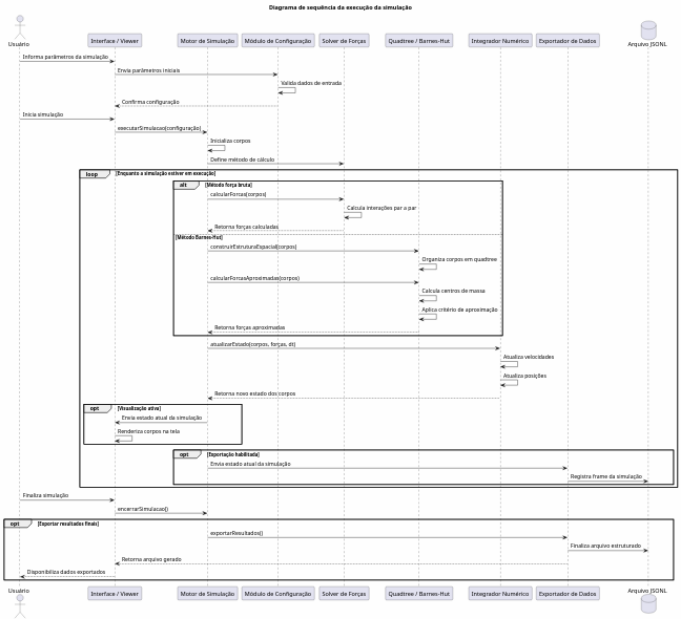


Figura 5. Diagrama de sequência. Fonte: autoria própria, 2026.

Durante a execução, o motor de simulação inicializa os corpos e define o método de cálculo de forças a ser utilizado. Caso seja selecionada a abordagem direta por força bruta, o solver calcula explicitamente as interações entre todos os pares de corpos. Caso seja selecionada a abordagem otimizada, o módulo de Barnes-Hut constrói a estrutura espacial, organiza os corpos em uma quadtree, calcula os centros de massa e aplica o critério de aproximação para estimar as forças exercidas por agrupamentos distantes.

Após o cálculo das forças, o integrador numérico atualiza o estado dos corpos, modificando suas velocidades e posições a cada passo temporal. Quando a visualização está ativa, o estado atualizado é enviado à interface, permitindo a renderização da evolução da simulação. Quando a exportação está habilitada, o estado de cada passo temporal é enviado ao exportador de dados, que registra as informações em um arquivo JSONL para análise posterior.

Por fim, ao encerrar a simulação, o sistema pode concluir a exportação dos resultados e disponibilizar ao usuário o arquivo estruturado gerado. Dessa forma, o diagrama evidencia a separação entre configuração, cálculo físico, atualização numérica, visualização e exportação, reforçando a proposta de uma arquitetura modular para a *engine*.

4.1.5. Protótipos

Para apresentar a ideia geral da *engine*, foram desenvolvidos protótipos das telas, componentes, menus e principais fluxos de uso do sistema. A prototipação auxilia na representação inicial da interface e permite visualizar a interação esperada antes da implementação completa da aplicação (Nielsen Norman Group,). Além disso, a elaboração das interfaces considera princípios gerais de projeto centrado no usuário, especialmente no que se refere à organização da interação entre usuário e sistema (International Organization for Standardization, 2019).

Os protótipos foram elaborados com o apoio da ferramenta *Excalidraw*, utilizada para criação de diagramas, esboços e representações visuais com estilo desenhado à mão (Excalidraw,). Inicialmente, as interfaces foram concebidas para uma aplicação local, podendo ser adaptadas posteriormente para uma versão *web*, conforme a evolução da proposta.

A Figura 6 apresenta dois estados possíveis da tela principal da aplicação.

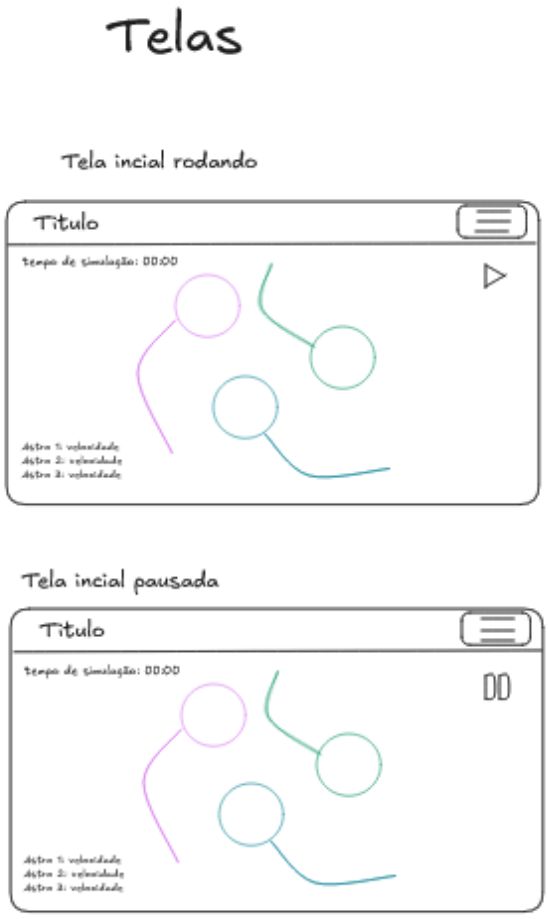


Figura 6. Protótipo das telas. Fonte: autoria própria, 2026.

Nas telas apresentadas, é possível observar o título da simulação, o botão de acesso ao menu hambúrguer, o botão de pausa e um *log* de eventos posicionado no canto inferior esquerdo. Esses elementos têm como objetivo permitir que o usuário acompanhe o estado da simulação e acesse as principais funcionalidades da aplicação de forma simples.

A Figura 7 apresenta os principais menus previstos para a interação do usuário com a aplicação.

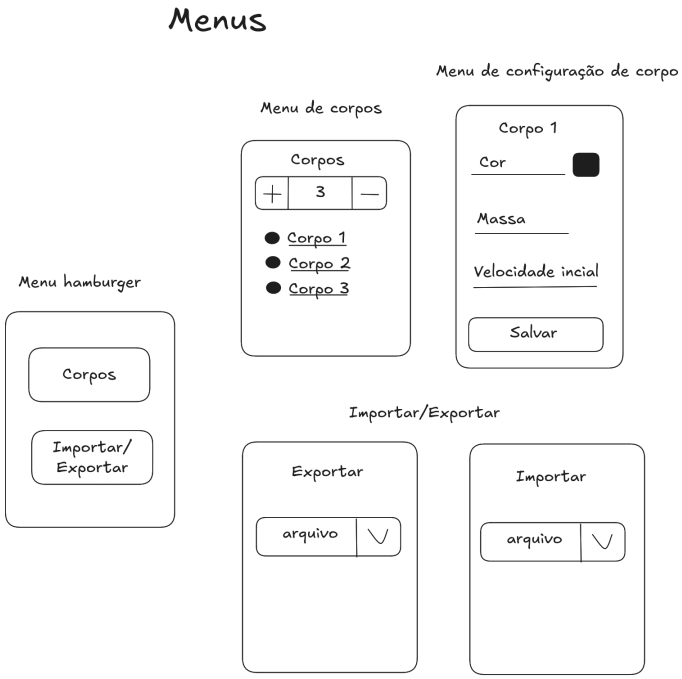


Figura 7. Protótipo dos menus. Fonte: autoria própria, 2026.

O primeiro menu representado é o menu hambúrguer, que permite o acesso ao menu de corpos e ao menu de exportação de arquivos. O menu de corpos permite adicionar novos corpos à simulação ou modificar corpos já existentes por meio de campos interativos. Ao selecionar um corpo, o menu de atualização é exibido, permitindo alterar propriedades como nome, cor, massa e velocidade inicial.

Também é previsto um menu de exportação, responsável por disponibilizar os dados da simulação em formato estruturado, como JSONL, permitindo sua análise posterior (JSON Lines, 2026). Além disso, o menu de importação permite carregar novamente uma simulação previamente gerada, possibilitando a reprodução ou continuidade de experimentos.

A Figura 8 apresenta dois dos principais fluxos de uso da aplicação: adicionar e configurar um corpo e exportar os dados de uma simulação.

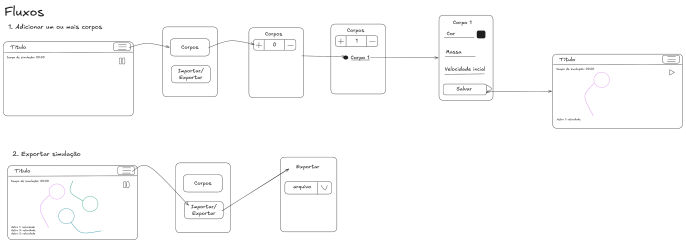


Figura 8. Protótipo dos fluxos. Fonte: autoria própria, 2026.

O primeiro fluxo consiste na adição de um novo corpo à simulação. Esse processo inicia na tela principal, passa pelo acesso ao menu hambúrguer, pela seleção da opção de

adicionar corpo e, posteriormente, pela edição das propriedades do corpo inserido, como massa, cor, nome e velocidade inicial.

O segundo fluxo representa a exportação dos dados da simulação. Após pausar ou finalizar a execução, a *engine* organiza as informações geradas em um formato estruturado e permite que o usuário exporte os dados para análise posterior. Esse fluxo reforça a proposta de separar a execução da simulação da etapa de análise dos resultados.

4.2. Arquitetura

A arquitetura proposta para o sistema é apresentada na Figura 9. Ela foi organizada de forma modular, separando a camada de visualização do núcleo responsável pela simulação física. Essa separação tem como objetivo permitir que a *engine* seja executada de forma independente da interface utilizada, favorecendo a reutilização do motor de simulação em diferentes ambientes de execução.

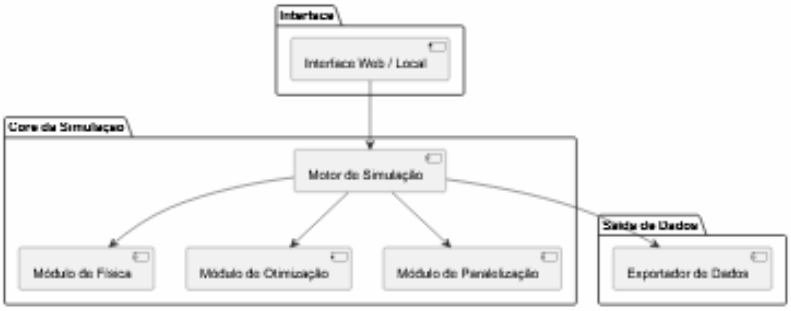


Figura 9. Diagrama de arquitetura. Fonte: autoria própria, 2026.

A estrutura geral da solução é composta por dois blocos principais: a camada de visualização e a *engine*. A camada de visualização é responsável pela interação com o usuário e pela apresentação dos resultados da simulação. Já a *engine* concentra a lógica principal do sistema, incluindo a configuração dos corpos, o cálculo das forças, a aplicação dos métodos de otimização, a atualização dos estados e a exportação dos dados gerados.

4.2.1. Visualização

A camada de visualização representa a interface por meio da qual o usuário poderá acompanhar a execução da simulação e observar a evolução do sistema de partículas ao longo do tempo. Inicialmente, essa visualização será desenvolvida em ambiente local, com foco na validação do comportamento físico da simulação e na análise visual dos resultados obtidos.

Como a arquitetura foi planejada de forma desacoplada, a visualização não será responsável pela lógica física da simulação. Seu papel será consumir os dados produzidos pela *engine* e apresentá-los de forma compreensível ao usuário. Essa decisão permite que, futuramente, a visualização local seja substituída ou complementada por outros formatos de interface, como um visualizador web ou um reprodutor de arquivos gerados pela simulação.

Além da apresentação visual, essa camada também poderá permitir a inserção de parâmetros iniciais, como quantidade de corpos, massas, posições, velocidades e método de cálculo utilizado. Dessa forma, a visualização atua como uma camada de entrada e de saída do sistema, sem comprometer a independência do núcleo de simulação.

4.2.2. Engine

A *engine* corresponde ao núcleo principal da aplicação, sendo responsável pela execução da simulação do problema de N-corpos. Sua organização será baseada em uma arquitetura *headless*, na qual o motor físico é desacoplado da camada de apresentação. Assim, o núcleo da simulação pode ser executado independentemente de uma interface gráfica específica, que possibilita maior reutilização e extensibilidade do sistema (Talon.One, 2026).

Dentro da *engine*, o módulo de simulação será responsável por coordenar o ciclo de execução, inicializar os corpos, controlar os passos temporais e acionar os demais componentes necessários ao processamento O módulo de cálculo de forças deverá implementar inicialmente a abordagem direta, baseada no cálculo explícito das interações gravitacionais entre todos os pares de corpos. Posteriormente, será integrada uma abordagem otimizada baseada em decomposição espacial hierárquica, utilizando *quadtree* e o algoritmo de Barnes-Hut para reduzir a complexidade computacional do cálculo das forças (BARNES; HUT, 1986; ArborJS, 2026; Princeton University, 2026).

A *engine* também deverá conter mecanismos para atualizar o estado dos corpos, considerando suas posições, velocidades e acelerações ao longo do tempo. Além disso, será responsável por disponibilizar os dados da simulação para análise posterior, por meio da exportação em formato estruturado, como JSON Lines (JSONL) (JSON Lines, 2026). Com isso, os resultados produzidos poderão ser armazenados, comparados e eventualmente consumidos por diferentes ferramentas de visualização ou análise.

Essa organização modular permite que novos métodos de cálculo, estratégias de otimização, formas de paralelização e interfaces de visualização sejam adicionados futuramente sem a necessidade de reescrever todo o sistema. Dessa forma, a arquitetura contribui diretamente para os objetivos de extensibilidade, reprodutibilidade e análise experimental definidos para o projeto.

Referências

Adriano Croco. *Analisando algoritmos: QuadTree*. 2026. Acesso em 20 jun. 2026. Disponível em: ⟨https://42bits.medium.com/analisando-algoritmos-quadtree-2fe86394f624⟩. Acesso em: 20 jun. 2026.

ArborJS. *Barnes-Hut Simulation Documentation*. 2026. Acesso em 22 mar. 2026. Disponível em: ⟨https://arborjs.org/docs/barnes-hut⟩. Acesso em: 22 mar. 2026.

BARNES, J.; HUT, P. A hierarchical O(N log N) force-calculation algorithm. *Nature*, v. 324, n. 6096, p. 446–449, 1986. Acesso em 20 jun. 2026. Disponível em: ⟨https://www.nature.com/articles/324446a0⟩. Acesso em: 20 jun. 2026.

BRANDT, A. *On Distributed Gravitational N-Body Simulations*. 2022. Acesso em 20 jun. 2026. Disponível em: ⟨https://arxiv.org/abs/2203.08966⟩. Acesso em: 20 jun. 2026.

Carnegie Mellon University. *N-Body Simulation*. 2026. Acesso em 17 mar. 2026. Disponível em: ⟨https://www.cs.cmu.edu/~scandal/alg/nbody.html⟩. Acesso em: 17 mar. 2026.

Excalidraw. *Virtual whiteboard for sketching hand-drawn like diagrams*. Disponível em: ⟨https://github.com/excalidraw/excalidraw⟩. Acesso em: 24 jun. 2026.

GREENGARD, L. The Numerical Solution of the N-Body Problem. *Computer Physics*, 1990. Acesso em 20 jun. 2026. Disponível em: ⟨hhttps://pubs.aip.org/aip/cip/article/4/2/142/509993/The-Numerical-Solution-of-the-N-Body-Problem⟩. Acesso em: 20 jun. 2026.

Instituto Nacional de Pesquisas Espaciais. *Simulações numéricas em sistemas gravitacionais*. 2026. Acesso em 23 mar. 2026. Disponível em: ⟨http://mtc-m16d.sid.inpe.br/col/sid.inpe.br/mtc-m19/2013/02.08.17.58/doc/publicacao.pdf⟩. Acesso em: 23 mar. 2026.

International Organization for Standardization. *ISO 9241-210:2019 Ergonomics of human-system interaction – Part 210: Human-centred design for interactive systems*. 2019. Disponível em: ⟨https://www.iso.org/standard/77520.html⟩. Acesso em: 24 jun. 2026.

JSON. *Introducing JSON*. 2026. Acesso em 30 maio. 2026. Disponível em: ⟨https://www.json.org/⟩. Acesso em: 30 maio 2026.

JSON Lines. *JSON Lines: Documentation for the JSON Lines text file format*. 2026. Acesso em 30 maio. 2026. Disponível em: ⟨https://jsonlines.org/⟩. Acesso em: 30 maio 2026.

KALE, M.; KRY, P. G. *Distributed Simulation of Large Multi-body Systems*. 2024. Acesso em 21 mar. 2026. Disponível em: ⟨https://arxiv.org/abs/2403.17261⟩. Acesso em: 21 mar. 2026.

Nielsen Norman Group. *Prototyping Articles, Videos, Reports, and Training Courses*. Disponível em: ⟨https://www.nngroup.com/topic/prototyping/⟩. Acesso em: 24 jun. 2026.

Odin. *Documentation*. 2026. Acesso em 30 maio. 2026. Disponível em: ⟨https://odin-lang.org/docs/⟩. Acesso em: 30 maio 2026.

Odin. *The Odin Programming Language*. 2026. Acesso em 30 maio. 2026. Disponível em: ⟨https://odin-lang.org/⟩. Acesso em: 30 maio 2026.

OLEGáRIO, M. V. T. *Desenvolvimento de simulação de n-corpos para o estudo da matéria escura*. 2024. Trabalho de Conclusão de Curso (Graduação) – Instituto de Física de São Carlos, Universidade de São Paulo, São Carlos. Disponível em: ⟨https://doi.org/10.11606/003205186⟩. Acesso em: 23 mar. 2026.

Princeton University. *Barnes-Hut Assignment*. 2026. Acesso em 24 mar. 2026. Disponível em: ⟨https://www.cs.princeton.edu/courses/archive/fall03/cs126/assignments/barnes-hut.html⟩. Acesso em: 24 mar. 2026.

REIN, H.; LIU, S.-F. *REBOUND: An open-source multi-purpose N-body code for collisional dynamics*. 2011. Acesso em 20 jun. 2026. Disponível em: ⟨https://arxiv.org/abs/1110.4876⟩. Acesso em: 20 jun. 2026.

SPRINGEL, V. The cosmological simulation code GADGET-2. *Monthly Notices of the Royal Astronomical Society*, v. 364, n. 4, p. 1105–1134, 2005. Acesso em 17 mar. 2026. Disponível em: ⟨https://arxiv.org/abs/astro-ph/0505010⟩. Acesso em: 21 mar. 2026.

Talon.One. *Headless commerce*. 2026. Acesso em 20 jun. 2026. Disponível em: ⟨https://www.talon.one/glossary/headless⟩. Acesso em: 20 jun. 2026.

TEUBEN, P. *NEMO Stellar Dynamics Toolbox*. 2026. Acesso em 20 jun. 2026. Disponível em: ⟨https://teuben.github.io/nemo/⟩. Acesso em: 20 jun. 2026.

University of North Carolina. *Proteus Publications*. 2026. Acesso em 17 mar. 2026. Disponível em: ⟨https://www.cs.unc.edu/Research/proteus/proteus-publications.html⟩. Acesso em: 17 mar. 2026.